

port Portal

- Adding a new ticket form
- Changing any triggers/automations relating to SSAT or Support KPIs

Criticality 3

These are changes with either no or very-low risk of negative impact, but where there is still some inherent complexity, or it is not fully automated and hands-off.

Some examples of Criticality 3 requests are:

- Adding a new form field on a Support form
- Bulk removing expired Zendesk organizations
- Adding a new Zendesk app that will make things more convenient for Support Engineers
- Removing or deactivating active macros

Criticality 4

These are changes that are exceedingly low risk and commonly executed, or which are fully automated. Often these will be changes that are mainly being recorded for visibility rather than as a substantial control measure.

Some examples of Criticality 4 requests are:

- Adding or removing users from a ZD organization
- Creating or updating macros

SECTION: security/customer-support-operations/coding-standards.md

^{Introduced via gitlab-com/support/support-ops/support-ops-project1963}

This document attempts to define coding standards and best practices to follow for the GitLab Support Operations team. By utilizing these guidelines, we ensure consistency, maintainability, simplicity, and better collaboration within the team. All contributors to our projects are expected to follow these guidelines, unless a specific exception is agreed upon.

General Principles

We believe the GitLab core values should guide all our coding decisions:

- Collaboration
 - Collaborate with teammates, review each other's code, and share knowledge.
 - Always be open to feedback and improve your code based on it.
 - Ensure your contributions are readable and understandable to others on the team
- Results
 - Focus on the impact and effectiveness of the code.
 - Prioritize working solutions over perfect solutions (but try to avoid shortcuts that undermine quality)

- Efficiency
 - Write clear and maintainable code.
 - Strive for simplicity without sacrificing functionality.
- Transparency
 - Code should be easy to understand and accessible to all.
 - Document assumptions, choices, and design decisions clearly when needed.
- Iteration
 - Code should be flexible and easy to refactor.
 - We aim for continuous improvement and don't shy away from updating or reworking code when necessary.

Naming Conventions

- Variables
 - Use descriptive names that clearly convey the purpose of the variable. If the name of the variable does not clearly convey the purpose, then add a comment on the code to explain it better.
 - All variable names should use snake case:
 - Examples: sumofpairings, gitlabuser
- Constants
 - Use descriptive names that clearly convey the purpose of the constant. If the name of the constant does not clearly convey the purpose, then add a comment on the code to explain it better.
 - Constants should be written in upper snake case:
 - Examples: MAXRETRIES, DEFAULTTIMEOUT
- Functions and Methods
 - Function and method names should describe the action they perform. If the name of the function/method does not clearly convey the purpose, then add a comment on the code to explain it better.
 - Function and method names should be written in snake case"
 - Examples: mynewfunction, checkgitlabuser
- Classes and Modules
 - Classes and modules should be written in PascalCase:
 - Examples: AccountBlocked, SupportSuperFormProcessor
- Filenames
 - All filenames should use snake case:
 - Examples: processaccount, compareonly

Code Formatting

- Indentation
 - Use 2 spaces per indentation level whenever possible.
 - Example:

```

ruby
def self.subscriptions
  @subscriptions ||= Readiness::GitLab::Namespaces.subscription(client, namespace)
end

```

- Line length
 - Keep lines of code to a maximum length of 80-120 characters (80 characters preferred). This ensures readability and allows code to fit comfortably in most environments.
- Blank lines
 - Use blank lines to separate functions, classes, and logically related code blocks
 - Use a single blank line between method definitions
- Spacing
 - Use a single space between operators (e.g. +, <, =, etc.)

Comments and Documentation

- Inline comments
 - Use inline comments sparingly. Only use them to clarify complex or non-obvious code. If you are needing a large number of comments, you are probably overcomplicating your code!
 - Keep comments up to date and relevant to the code they describe.

Code Complexity

- Avoid deeply nested code. Refactor code to use helper functions or early returns if nesting becomes too complex.
- Limit function size. Each function should have a clear, single responsibility and be relatively small (under 30 lines).

Testing

Always aim for high test coverage! When in doubt, test your code thoroughly!

Version control

- Commit messages
- Branches
 - The format of a branch should follow the format of USERNAME-PROJECT-IID, where:
 - USERNAME is your gitlab.com username
 - PROJECT is the project's slug
 - IID is the issue's IID
 - Example:
 - For branch jcolyer is creating for the work of issue <https://gitlab.com/gitlab-com/support/support-ops/support-ops-project/-/issues/1963>, your branch name should be jcolyer-support-ops-project-1963
 - The exception to this is branches for the ruby gems, which should always use the format vX.X.YY.ZZ, where XX.YY.ZZ is the new semantic version you are working on.

Use semantic versioning

Whenever you need to use version numbers, you should strive to use semantic versioning, which is in the format of MAJOR.MINOR.PATCH. When increasing the version number, you should use the following to determine which number to increase:

- Increase the MAJOR if you do a sizable refactor

- Increase the MINOR if you add or remove functionality
- Increase the PATCH if you are making small changes (wording changes, bug fixes, etc.)

Remember, when increasing a value:

- If increasing MAJOR, the new values of MINOR and PATCH are 0
- If increasing MINOR, the new value PATCH are 0 (and MAJOR remaining unchanged)
- If increasing PATCH, the values of MAJOR and MINOR remain unchanged

To help you, here are some examples:

Starting Version	MAJOR update	MINOR update	PATCH update
1.0.0	2.0.0	1.1.0	1.0.1
1.9.127	2.0.0	1.10.0	1.9.128
2.99.0	3.0.0	2.100.0	2.99.1
9.99.9	10.0.0	9.100.0	9.99.10

Using semantic versioning when only two numeric values are allowed

If you are working with something only allowed two numeric values (such as 1.01 or 9.8), you would instead combine the definitions of MINOR and PATCH for the second value. This results in the needed format of xx.yy and allows you to maintain a close semblance to semantic versioning.

Thus, when increasing the version number, you should use the following to determine which number to increase:

- Increase the xx if you do a sizable refactor
- Increase the yy if you add or remove functionality, or if you are making small changes (wording changes, bug fixes, etc.)

To help you, here are some examples:

Starting Version	MAJOR update	MINOR/PATCH update
1.0	2.0	1.1
1.9	2.0	1.10
2.99	3.0	2.100
9.99	10.0	9.100

Gem CHANGELOGs

When working on ruby gems, you should maintain a CHANGELOG.md file. This should detail the changes you are using. This will be used in the gem's documentation. While it can vary from gem to gem, a good format for an entry to use is:

markdown

vXX.YY.ZZ - YEAR-MONTH

- Detail of change

- Detail of change
- Detail of change

Where:

- XX is the MAJOR version of the semantic versioning
- YY is the MINOR version of the semantic versioning
- ZZ is the PATCH version of the semantic versioning
- YEAR is the 4 digit year (e.g. 2025)
- MONTH is the 2 digit month (e.g. 08)

Error Handling

Whenever possible, have your code gracefully handle errors. Where this is not possible, ensure the errors clearly explain what the problem is (and bonus points if they tell you what to do about it).

Performance Considerations

- Focus on performance only after your code is functional and maintainable. Avoid premature optimization.
- Measure performance impacts using profiling tools and identify bottlenecks before optimizing.
- Use common sense to reduce API calls to our tooling.
 - Example: While you could manually fetch every ticket from Zendesk one by one, there is no logical reason to when the Readiness Gem will do this efficiently using the list method.

Security

- Validate all inputs to prevent injection attacks (e.g., SQL Injection, XSS).
- Never store passwords, tokens, etc. in plaintext. Use CI/CD variables.

Code Reviews

- All code must undergo peer review before merging. Ensure reviews are thorough and address quality, consistency, and security.
- Embrace constructive feedback and improve the code collaboratively.

Language specific guidelines

Ruby

- Whenever possible, utilize the GitLab Support Readiness Gem
- Whenever possible, try to ensure your code presents no issues when run via the rubocop linter.

Examples to help you get started

Writing a ruby script

When writing a ruby script, you should always start it with:

```
ruby
#!/usr/bin/env ruby
frozenstringliteral: true

require 'supportreadiness'
```

This ensures you can start coding on the very next line and have all the dependencies you might need.

Connecting to Zendesk via ruby

When you need to connect to Zendesk, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'

zendeskconfig = Readiness::Zendesk::Configuration.new
zendeskconfig.username = ENV.fetch('ZDUSERNAME')
zendeskconfig.token = ENV.fetch('ZDTOKEN')
zendeskconfig.url = ENV.fetch('ZDURL')
zendeskclient = Readiness::Zendesk::Client.new(zendeskconfig)
```

For the above example, the environment variables translate to:

- ZDUSERNAME: The email address of the user making the connection. Take note you do not put /token or anything at the end (the gem will handle this for you).
- ZDTOKEN: The API token being used to connect to Zendesk
- ZDURL: The API URL. This should be one of the following values:
 - Zendesk Global: <https://gitlab.zendesk.com/api/v2>
 - Zendesk Global Sandbox: <https://gitlab1707170878.zendesk.com/api/v2>
 - Zendesk US Government: <https://gitlab-federal-support.zendesk.com/api/v2>
 - Zendesk US Government Sandbox: <https://gitlabfederalsupport1585318082.zendesk.com/api/v2>

Connecting to GitLab.com via ruby

When you need to connect to gitlab.com, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'

gitlabconfig = Readiness::GitLab::Configuration.new
gitlabconfig.token = ENV.fetch('GLTOKEN')
gitlabclient = Readiness::GitLab::Client.new(gitlabconfig)
```

For the above example, the environment variables translate to:

- GLTOKEN: The API token being used to connect to gitlab.com

Connecting to Slack via ruby

When you need to connect to Slack, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'

slackconfig = Readiness::Slack::Configuration.new
slackconfig.url = ENV.fetch('SLACKURL')
slackclient = Readiness::Slack::Client.new(slackconfig)
```

For the above example, the environment variables translate to:

- SLACKURL: The webhook URL to use in the connection

Connecting to Redis via ruby

When you need to connect to Redis, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'

redisclient = Readiness::Redis.new(
  ENV.fetch('REDISHOST'),
  ENV.fetch('REDISPORT'),
  ENV.fetch('REDISPASSWORD')
)
```

For the above example, the environment variables translate to:

- REDISHOST: The hostname used to connect to Redis
- REDISPASSWORD: The password used to connect to Redis
- REDISPORT: The port used to connect to Redis

Connecting to Salesforce via ruby

When you need to connect to Salesforce, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'
```

```
salesforceconfig = Readiness::Salesforce::Configuration.new
salesforceconfig.apiversion = '58.0'
salesforceconfig.clientid = ENV.fetch('SFDCCLIENTID')
salesforceconfig.clientsecret = ENV.fetch('SFDCCLIENTSECRET')
salesforceconfig.password = ENV.fetch('SFDCPASSWORD')
salesforceconfig.securitytoken = ENV.fetch('SFDCSECURITYTOKEN')
salesforceconfig.username = ENV.fetch('SFDCUSERNAME')
salesforceclient = Readiness::Salesforce::Client.new(salesforceconfig)
```

For the above example, the environment variables translate to:

- SFDCCLIENTID: The client ID used to connect to Salesforce
- SFDCCLIENTSECRET: The client secret token used to connect to Salesforce
- SFDCHOST: The hostname used to connect to Salesforce
- SFDCPASSWORD: The password used to connect to Salesforce
- SFDCSECURITYTOKEN: The security token used to connect to Salesforce
- SFDCUSERNAME: The username used to connect to Salesforce

Connecting to Mailgun via ruby

When you need to connect to Mailgun, utilize the GitLab Support Readiness Gem by doing the following:

```
ruby
require 'supportreadiness'
```

```
mailgunconfig = Readiness::Mailgun::Configuration.new
mailgunconfig.domain = ENV.fetch('MAILGUNDOMAIN')
mailgunconfig.token = ENV.fetch('MAILGUNKEY')
mailgunclient = Readiness::Mailgun::Client.new(mailgunconfig)
```

For the above example, the environment variables translate to:

- MAILGUNDOMAIN: The domain within Mailgun to use
- MAILGUNKEY: The API token being used to connect to Mailgun

SECTION: security/customer-support-operations/division-of-responsibilities.md

To help ensure the team doesn't get overwhelmed and has the ability to focus on specialization and learning, we divide out the responsibilities among our team.

Due to the complex nature of our system criticality, we maintain it via the Customer Support Operations Division of Responsibilities Google sheet (internal access only).

SECTION: security/customer-support-operations/frequently-asked-questions.md

How can I quickly check the status of the various Customer Support Operations components?

You can use our status page to check on the status our component.

If we receive any problem in using Zendesk, can we contact Zendesk directly?

Please contact Customer Support Operations team first. Discuss the problem by asking a question in channel and tagging @support-ops. It is a high probability that we can help you resolve the problem at hand. In cases where we cannot and we do need to contact Zendesk support directly, it is best to have Customer Support Operations handle that.

What will happen if Zendesk is down globally?

Zendesk will only go down when the internet is globally affected because they use Pods for services. This ensures that if a region is facing downtime, Zendesk can quickly mitigate that while making sure services run smoothly. However, if you are still facing any problem accessing Zendesk, please contact the Customer Support Operations team. In the case that Zendesk is down globally, we have email support option available.

Is there any disaster recovery plan available?

Zendesk keeps the data in backup servers with all due diligence. This ensures that we can recover data when it is needed. These backups are utilized to restore Zendesk in the case it fails due to a problem on Zendesk's end.

Also, the Customer Support Operations team ensures all triggers, automations, views, macros, forms, fields, conditions, etc are documented to save the hassle of writing up everything from scratch.

What if I want access to something owned by Customer Support Readiness?

Following with Security's Access Management Standards, we follow the principle of least privilege. As such, we review each request on a case by case basis.

To begin the process, file an Individual Bulk Access Request issue. Once approved by your manager, assign it to @jcolyer and @dtragjasi, who will review the situation and determine how to best proceed.

When using Zendesk, the aspect ratio is off

(Such as only showing one tab before using group, odd blank spaces appearing, etc.)

While the exact cause has not yet been determined, the spacing and grouping used in Zendesk is dependent on the aspect ratio and screen size of your monitor. When issues surrounding it have occurred, we have found Zooming in and back out (or zooming out and back in) often fixes the problem. We have also found doing a hard refresh (Ctrl+Shift+R / Cmd+Shift+R) can rectify the problem.

Failing all of that working, the suggested fix would be to log out, clear your cache, clear your cookies, and log back in.

SECTION: security/customer-support-operations/docs/_index.md

SECTION: security/customer-support-operations/docs/recommended-setup.md

{{% alert title="Note" color="primary" %}}

These are all recommendations. Some of this might be required for doing your job (1Password, Zoom, Slack, Okta, etc.), but largely if you feel you have a better alternative, go for it!

{{% /alert %}}

What tooling do we recommend

| Tooling | Why we recommend it |

|-----|-----|

| 1Password | You are going to be logging into things. This is helpful to have handy |

| .bashrc/.bashprofile | This will make running a lot of our scripts via CLI easy. See the setup for more information |

| GitLab Token | Will help you in running API calls via CLI to gitlab.com |

| Homebrew | This is super useful for installing things on macOS |

| nodenv | We work with a lot of projects that use a variety of node versions. This tool helps you switch between them very quickly. |

| Okta browser plugin | You are going to be logging into things. This is helpful to have handy |

| Openshot Video editor | We often make videos for training/documentation/etc. Having a solid video editor handy is useful. Openshot is easy to use and open source. |

| rbenv | We work with a lot of projects that use a variety of ruby versions. This tool helps you switch between them very quickly. |

| Slack | How a good portion of the company communicates |

| Visual Studios Code | You are going to need a code editor. This one aligns with the gitlab WebIDE |

| Zendesk Global Token | You are going to be using the Zendesk API a good bit. Having your own admin token will be very useful |

| Zendesk Global Sandbox Token | You are going to be using the Zendesk API a good bit. Having your own admin token will be very useful |

| Zendesk US Federal Token | You are going to be using the Zendesk API a good bit. Having your own admin token will be very useful |

| Zendesk US Federal Sandbox Token | You are going to be using the Zendesk API a good bit. Having your own admin token will be very useful |

| Zoom | What we use 99% of the time for meetings |

Homebrew

```
{{% alert title="Note" color="primary" %}}
```

This is only for those using macOS

```
{{% /alert %}}
```

To install homebrew, simply use their setup script:

```
bash
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

rbenv

This is a fantastic tool, especially when needing to utilize many different ruby versions! We highly recommend installing and setting this up on your station.

Once you have this setup, we recommend the following commands be used to install the more common ruby versions we use:

- rbenv install 3.0.1
- rbenv install 3.2.2

macOS setup for rbenv

This is easily done using Homebrew. The command for this is:

```
brew install rbenv
```

If you ever need to upgrade it (to get newer ruby versions), you simply would do:

```
brew upgrade rbenv ruby-build
```

Linux setup for rbenv

You can use a package manager (such as apt) for this, however we would recommend installing this via git instead. To do this, run the following commands:

```
bash
git clone https://github.com/rbenv/rbenv.git ~/.rbenv
echo 'export PATH="$HOME/.rbenv/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(rbenv init - bash)"' >> ~/.bashrc
source ~/.bashrc
mkdir -p "$(rbenv root)/plugins
git clone https://github.com/rbenv/ruby-build.git "$(rbenv root)/plugins/ruby-build
source ~/.bashrc
curl -fsSL https://github.com/rbenv/rbenv-installer/raw/main/bin/rbenv-doctor | bash
```

To upgrade rbenv and the plugin, you would simply do the following:

```
bash
cd ~/.rbenv
git pull
cd ~/.rbenv/plugins/ruby-build
git pull
```

1Password

There are two ways to go about this:

- App based (macOS only)
- Browser based

App based setup for 1Password

```
{{% alert title="Note" color="primary" %}}
```

This is for macOS only. There is a Linux variant, however it does not currently work properly.

```
{{% /alert %}}
```

To install the 1Password app, you would utilize Homebrew via the following command:

```
bash
brew install --cask 1password
```

To upgrade it, you would use:

```
bash
brew upgrade --cask 1password
```

Browser based setup for 1Password

You can add this to your Chrome based browser using the 1Password browser plugin for Chrome page.

You can add this to your Firefox based browser using the 1Password browser plugin for Firefox page.

Okta browser plugin

You can add this to your Chrome based browser using the Okta browser plugin for Chrome page.

You can add this to your Firefox based browser using the Okta browser plugin for Firefox page.

Once installed, you simply ensure you are logged into Okta via your browser and it should begin working.

nodenv

This is a fantastic tool, especially when needing to utilize many different node versions! We highly recommend installing and setting this up on your station.

macOS setup for nodenv

This is easily done using Homebrew. The command for this is:

```
brew install nodenv
```

If you ever need to upgrade it (to get newer node versions), you simply would do:

```
brew upgrade nodenv node-build
```

Linux setup for nodenv

You can use a package manager (such as apt) for this, however we would recommend installing this via git instead. To do this, run the following commands:

```
bash
git clone https://github.com/nodenv/nodenv.git ~/.nodenv
echo 'export PATH="$HOME/.nodenv/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(nodenv init -)"' >> ~/.bashrc
source ~/.bashrc
mkdir -p "$(nodenv root)"/plugins
git clone https://github.com/nodenv/node-build.git "$(nodenv root)"/plugins/node-build
source ~/.bashrc
curl -fsSL https://github.com/nodenv/nodenv-installer/raw/master/bin/nodenv-doctor | bash
```

To upgrade nodenv and the plugin, you would simply do the following:

```
bash
cd ~/.nodenv
git pull
cd ~/.nodenv/plugins/node-build
git pull
```

Openshot Video Editor

This one is only really needed if you need a video editor and do not have a preference. Several of us at GitLab use this and find it very simple to use.

macOS setup for Openshot

This is easily done using Homebrew. The command for this is:

```
bash
brew install --cask openshot-video-editor
```

If you ever need to upgrade it, you simply would do:

```
bash
brew upgrade --cask openshot-video-editor
```

Linux setup for Openshot

As this is not part of normal package repositories, you'll need to install a PPA first:

```
bash
sudo add-apt-repository ppa:openshot.developers/ppa
sudo apt-get update
sudo apt-get install openshot-qt
```

.bashrc/.bashprofile/.zshrc setup

What exactly you put in this file it really up to you, but to help ensure all the scripts and the like we use work properly, we would at least recommend doing the following:

```
bash
us fed ZD instance
export USZDURL='https://gitlab-federal-support.zendesk.com/api/v2'
export USZDUSERNAME='YOUREMAILADDRESS'
export USZDTOKEN='ZENDESKUSFEDERALADMINTOKEN'
```

```
main ZD instance
export ZDURL='https://gitlab.zendesk.com/api/v2'
export ZDUSERNAME='YOUREMAILADDRESS'
export ZDTOKEN='ZENDESKGLOBALADMINTOKEN'
```

```
main sandbox
export ZDSBURL='https://gitlab1707170878.zendesk.com/api/v2'
export ZDSBUSERNAME='YOUREMAILADDRESS'
export ZDSBTOKEN='ZENDESKGLOBALSANDBOXADMINTOKEN'
```

```
us fed sandbox
export USSBZDURL='https://gitlabfederalsupport1585318082.zendesk.com/api/v2'
export USSBZDUSERNAME='YOUREMAILADDRESS'
export USSBZDTOKEN='ZENDESKUSFEDERALSANDBOXADMINTOKEN'
```

```
GitLab token
export GLTOKEN='YOURGITLABTOKEN'
```

Zendesk tokens

As an admin for Zendesk, you are very likely to need to make API calls. Doing this via an API token is useful (and how many of our scripts do it). As such, it is recommend you generate an API token for yourself on every Zendesk instance (production and sandbox) you will be managing.

GitLab token

We often make things that interact with gitlab.com. As such, having an API token handy for API calls is always useful!

Slack

This is going to be required for your job, as we communicate a good bit via Slack.

Slack setup for macOS

This is easily done using Homebrew. The command for this is:

```
bash
brew install --cask slack
```

If you ever need to upgrade it, you simply would do:

```
bash
brew upgrade --cask slack
```

Slack setup for Linux

To do this, you will want to install the DEB package from the Slack download page and then install the DEB file like so:

```
bash
sudo dpkg -i /absolute/path/to/deb/file
sudo apt-get install -f
```

Visual Studios Code

This one is only really needed if you need a code editor and do not have a preference. Several of us at GitLab use this and it aligns with the GitLab WebIDE.

Visual Studios Code setup for macOS

This is easily done using Homebrew. The command for this is:

```
bash
```

```
brew install --cask visual-studio-code
```

If you ever need to upgrade it, you simply would do:

```
bash
brew upgrade --cask visual-studio-code
```

Visual Studios Code setup for Linux

To do this, you will want to install the DEB package from the Visual Studios Code download link and then install the DEB file like so:

```
bash
sudo dpkg -i /absolute/path/to/deb/file
sudo apt-get install -f
```

Zoom

This is going to be required for your job, as we communicate a good bit via Zoom.

Zoom setup for macOS

This is easily done using Homebrew. The command for this is:

```
bash
brew install --cask zoom
```

If you ever need to upgrade it, you simply would do:

```
bash
brew upgrade --cask zoom
```

Zoom setup for Linux

To do this, go to the Zoom download center and download the DEB file. Then install it using the following commands:

```
bash
sudo dpkg -i /absolute/path/to/deb/file
sudo apt-get install -f
```

Filesystem setup

We utilize a large number of repositories. While you can do many things in the GitLab UI, you

are going to eventually run into the need to utilize command line.

As such, we recommend two things:

1. Have local clones of all the repos we work with (that you have access to)
1. Have a script to mass update them as needed

The exact way you go about this is up to you.

```
{{% alert title="How Jason does it" color="primary" %}}
```

```
<details>
```

```
<summary>Expand for a wall of text</summary>
```

I will preface this by stating this is solely how I prefer to do it. This is all going to be based around preference. But you ever see me do things via CLI, matching what I do might help you in replication.

Firstly, I made a folder in my home directory named dev

I think begin making child folders to match the namespace pathing of all the repos I deal with.

Since I'm often working out of our namespace and the gitlab-com namespace, I made two folders in dev:

```
plaintext
```

```
.
```

```
... dev
```

```
... gitlab-com
```

```
... gitlab-support-readiness
```

From here, I followed the natural path of our namespace pathing. So as an example, the pathing to the Zendesk Global ZenDuo app on my filesystem looks like this:

```
plaintext
```

```
.
```

```
... dev
```

```
... gitlab-support-readiness
```

```
... zendesk-global
```

```
... apps
```

In that apps folder, I then did the git clone of the repo in question.

Once I had everything cloned and setup, I then made a script to update my repos (and any submodules). I tend to run this at the start of my day. It helps prevent me from working out of a stale repo.

```
bash
```

```
#!/bin/bash
```

```
echo 'Updating Customer Support Operations repos...'
CWD=$PWD
find /home/jason/dev/gitlab-support-readiness -type d -name .git | while read dir; do
    path=$(echo $dir | sed s/"\.git$")
    echo "  Updating $path"
    cd $path
    git submodule update --remote --quiet 2>/dev/null
    git pull -q 2>/dev/null
    if [ $? -eq 1 ]; then
        echo "    Branch issue: $(git branch|grep " ")"
    fi
done
cd $CWD
```

As I added other paths (like for the support managed content, the handbook, etc.), I modified that update script to ensure it included those as well.

Finally, I made some aliases in my .bashrc file to help me with quicker navigation:

```
bash
alias cdops='cd /home/jason/dev/gitlab-support-readiness'
alias cdglobal='cd /home/jason/dev/gitlab-support-readiness/zendesk-global'
alias cdusfed='cd /home/jason/dev/gitlab-support-readiness/zendesk-us-government'
alias cdusgov='cd /home/jason/dev/gitlab-support-readiness/zendesk-us-government'
alias cdforms='cd /home/jason/dev/gitlab-support-readiness/forms'
alias cdprocessor='cd /home/jason/dev/gitlab-support-readiness/processors'
alias cdslack='cd /home/jason/dev/gitlab-support-readiness/slack'
alias cdteam='cd /home/jason/dev/gitlab-support-readiness/support-team'
alias cdhandbook='cd /home/jason/dev/gitlab-com/content-sites/handbook'
alias cdgem='cd /home/jason/dev/gitlab-support-readiness/gitlabsupportreadinessgem'
```

I also tweaked my shell to display the branch I am working in on the prompt, mainly to help me keep track of it. This is also done in the .bashrc file (not this is going to be specifically for Linux as that is what I use):

```
bash
Show branch info on PS1
```

```
parsegitbranch() {
    git branch 2> /dev/null | sed -e '/^[^/]d' -e 's/ \(.)/ (\1)/'
}
```

```
export PS1="\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\]\[\e[91m\]\$(parsegitbranch)\[\e[00m\]$ "
```

</details>

{{% /alert %}}

Useful links

- Homebrew site
- rbenv repo
- Calendly browser plugin for Chrome
- Calendly browser plugin for Firefox
- 1Password browser plugin for Chrome
- 1Password browser plugin for Firefox
- Okta browser plugin for Chrome
- Okta browser plugin for Firefox
- Slack download page
- Zoom download center

SECTION: security/customer-support-operations/docs/support-super-form-processor.md

^{Introduced via support-team-meta5654}

{{% pageinfo color="warning" %}}

This is an information page for the Support Super form.

If you are looking for information about managing it, please see workflows.

{{% /pageinfo %}}

While there are many functions here, most are involving creating artifacts for other teams to work (such as issues or tickets). This page will focus on the ones that directly (or indirectly) involve Customer Support Operations.

Set the collaboration project ID for an organization

This automatically will set the values on the corresponding organization to specify a collaboration project ID.

Set an Assigned Support Engineer for an organization

This automatically will set the values on the corresponding organization to ensure auto assignment is done for tickets filed by said organization (if they have entitled to an ASE).

Enable US Government Support exception for an Account

This is for enabling US Government Support on a SFDC account that would not normally be entitled to it. Please note the account must have already purchased the required US Government SKU.

It will create a request issue.

Disable US Government Support exception for an Account

This is for disabling US Government Support on a SFDC account that currently has an exception on it.

It will create a request issue.

Create a Zendesk macro

This will handle macro creation, either via automation or via a request issue. For "simple" macros, it fully creates the macro automatically. For "complex" ones, it will create a request issue.

Edit a Zendesk macro

This will handle macro edit requests by creating a request issue.

Create a Zendesk article

This will automatically handle the creation of Zendesk articles (for the corresponding instances it needed for).

Check namespace availability

This is for checking the availability of a namespace. In the event the namespace is in use, this request will also check into the potential of the namespace being releasable. Please note this is fully automated. The request will create an issue that has the results of the check posted.

SECTION: security/customer-support-operations/docs/sync-repo-files.md

To manage the many components of Zendesk we work with, we utilize sync repos. These enable use to have version controlled projects containing all the data and information that makes out our Zendesk configuration.

With this, we end up with a YAML file per item (one per trigger, one per macro, etc.).

These files contain all the relevant information for that item (minus the ID value). We also strive to keep them as human-readable as possible (so it is easy to read and code).

Auto-conversion

Through the use of the `gitlabsupportreadiness` gem, we have several auto-conversions that occur when parsing the YAML files of our sync repos. This is to help keep things as "human-readable" as possible (and make it so we don't have to hard code IDs into our repos).

Conditions

Applicable to:

- Automations
- SLA Policies (called filters, but same thing)
- Triggers
- Views

Normal Field value	Key to edit	Text to use
-----	-----	-----
assigneeid	value	The user's email address
brandid	value	The name of the brand
customfields	field	Field: xxx, where xxx is the field's title attribute
groupid	value	The name of the group
organizationid	value	The organization's salesforceid value
requesterrole	value	Agent, Light Agent, End User, Admin
role	value	Agent End User, or the user's email address
satisfactionreasoncode	value	The name of the satisfaction reason
scheduleid	value	Schedule: xxx, where xxx is the schedule's name attribute (or " if using a blank)
ticketformid	value	Form: xxx, where xxx is the form's name attribute
viaid	value	The name of the via method (see here)

Actions

Applicable to:

- Automations
- Macros
- Triggers

Normal Field value	Key to edit	Text to use
-----	-----	-----
assigneeid	value	The user's email address
brandid	value	The name of the brand
customfields	field	Field: xxx, where xxx is the field's title attribute
groupid	value	The name of the group
notificationtarget	value	The target's title attribute
notificationwebhook	value	The webhook's name attribute
organizationid	value	The organization's salesforceid value
role	value	Agent End User, or the user's email address
setschedule	value	Schedule: xxx, where xxx is the schedule's name attribute (or " if using a blank)
ticketformid	value	Form: xxx, where xxx is the form's name attribute

View specific

The only view specific one is the restrictions attribute. You would use the group's name or the user's email address (instead of an ID value).

Form specific

- For the ticketfieldids attribute, use the field's title attribute
- For the enduserconditions and agentconditions attributes:
 - Use the field's title attribute for the parentfieldid value
 - Use the field's title attribute for the id value under the childfields attribute

Themes

+All hbs files within the templates folder and the script.js file are parsed by our sync repos to handle the conversion of ticket form names and ticket field titles to their corresponding ID value.

+For ticket forms, you use the format `[[form: NAMEOFFORM]]`. As an example, for the Zendesk Global Sandbox, using `[[form: Support Ops]]` would be converted to 12510630404508

+For ticket fields, you use the format `[[field: TITLEOFFORM]]`. As an example, for the Zendesk Global Sandbox, using `[[field: Customer Priority]]` would be converted to 12510561670940.

Apps

All js files and all the html file are parsed by our sync repos to handle the conversion of ticket form names and ticket field titles to their corresponding ID value.

For ticket forms, you use the format `[[Form: NAMEOFFORM]]`. As an example, for the Zendesk Global Sandbox, using `[[Form: Support Ops]]` would be converted to 12510630404508

For ticket fields, you use the format `[[Field: TITLEOFFORM]]`. As an example, for the Zendesk Global Sandbox, using `[[Field: Customer Priority]]` would be converted to 12510561670940.

SECTION: security/customer-support-operations/docs/system-checkers.md

Calendly

^{Introduced via [gitlab-com/support/support-ops/support-ops-project1892](https://gitlab.com/support/support-ops/support-ops-project1892)}

Handles checking and notifying about the status for the Calendly system. Works primarily off of RSS feed for <https://www.calendlystatus.com>. This scrapes the information from that RSS feed once per hour to look for new incidents and maintenance events.

The project is located at
gitlab-support-readiness/system-checkers/calendly

GitLab

^{Introduced via [gitlab-com/support/support-ops/support-ops-project1895](https://gitlab.com/support/support-ops/support-ops-project1895)}

Handles checking and notifying about potential pipelines issues on important projects we utilize on gitlab.com. This checks for an abnormal amount of pending jobs (50 or more) and abnormally long running jobs (25 minutes or longer).

The project is located at
gitlab-support-readiness/system-checkers/gitlab

Pagerduty

^{Introduced via gitlab-com/support/support-ops/support-ops-project1891}

Handles checking and notifying about:

- the status for the Pagerduty system
- non-accessible Support schedules
- non-accessible Support services
- non-accessible Support escalation policies

Works off of the Pagerduty API and webhooks sent from
<https://status.pagerduty.com/>

The project is located at
gitlab-support-readiness/system-checkers/pagerduty

Unbabel

^{Introduced via gitlab-com/support/support-ops/support-ops-project1893}

Handles checking and notifying about the status for the Unbabel system. Works off of the Unbabel status API to check for information about incidents.

The project is located at
gitlab-support-readiness/system-checkers/unbabel

Zendesk Global

^{Introduced via gitlab-com/support/support-ops/support-ops-project1889}

Handles checking and notifying about the status for the Zendesk Global system. Works off of the Zendesk status API to check for information about incidents and maintenances.

The project is located at
gitlab-support-readiness/system-checkers/zendesk-global

Zendesk US Government

^{Introduced via gitlab-com/support/support-ops/support-ops-project1890}

Handles checking and notifying about the status for the Zendesk US Government system. Works off of the Zendesk status API to check for information about incidents and maintenances.

The project is located at
gitlab-support-readiness/system-checkers/zendesk-us-government

SECTION: security/customer-support-operations/docs/system-criticality.md

Due to the complex nature of our system criticality, we maintain it via the Customer Support Operations System Criticality Google sheet (internal access only).

SECTION: security/customer-support-operations/docs/team_member_licenses.md

^{Introduced via support-team-meta3335}

The process

The process for these is:

1. The GitLab team member submits a request via the Support Super Form.
1. If valid, an issue is filed in support-ops-project.
1. Support Operations logs into the staging cDot app and confirms there is not an already existing license.
1. Support Operations generates the license using the parameters in the issue.
1. Support Operations updates the issue and closes it out.

Checking for an existing license

1. Login to the staging cDot app via Okta.
1. Click the Licenses link on the left-hand side of the page.
1. Type the email address in the search box and click Refresh.
1. Review the results to see if an existing license (that is not expired) exists can still be utilized.
1. If one does exist, please ask for clarification why a new license is needed when an existing one can be used still.
1. Review the clarification given and make a decision if a new license is warranted.

Creating a license

Note Before proceeding, please confirm the requested end date on the issue does not exceed 1 year. If it does, we cannot proceed with the request until the requester changes that.

1. Login to the staging cDot app via Okta.
1. Click the Licenses link on the left-hand side of the page.
1. Click the + Add new License at the top of the page.
1. Fill out the details on the form
1. Click the · Save button at the bottom of the form.

Field	Value
License Type	Legacy License
Name	value from the issue
Company	GitLab - Team Member License
Email	value from the issue
Zuora subscription ID	leave this blank
Zuora subscription name	leave this blank
Users count	value from the issue
Previous users count	0 or value from the issue
Trueup count	0 or value from the issue
Plan code	value from the issue
Trial	leave unchecked
Start date	today's date
End date	value from the issue
Notes	the issue link

Updating the issue

After you have generated the license, you need to reply and close out the issue.

To do this, reply with the following:

- > The staging license has been generated at this time. Search for the subject
- > Your GitLab License File in your e-mail to locate it.
- >
- > For information on applying a license, please read through
- > our documentation.
- >
- > As this is a staging license, please keep in mind you must setup your GitLab
- > deployment to work with those.

>
> - For non-GDK instances, please see
> our documentation.
> - For GDK instances:
> - Ensure that you have a env.runit file created in the root directory of
> your GDK repository.
> - It should contain the following contents:
>
> bash
> export GITLABLICENSEMODE=test
> export CUSTOMERPORTALURL="https://customers.staging.gitlab.com"
>
>
> - After the env.runit file is setup, please run gdk restart
> - You should then be able to apply the license to your GDK instance
>
> If you get an error that your license key is invalid, this will indicate you
> have not done the needed setup to work with a staging license.

After sending that reply, do the following:

- Add time spent on the issue
- Add the label ~Readiness::Completed to the issue
- Close the issue

SECTION: security/customer-support-operations/docs/calendly/_index.md

{{% pageinfo color="info" %}}

This is an informational page for Calendly. It may not reflect the way we actually manage it.

If you are looking for information about maintaining it, please see our workflows

{{% /pageinfo %}}

What is Calendly

Calendly is an online appointment scheduling tool the GitLab Support team uses for scheduling customer calls.

User management

User management in Calendly is primarily done via the Admin center's Users page. To access this page after logging in, you would click navigate to the Admin center by clicking the option at the lower-left of the page. After navigating there, click Users on the left-hand side to get to the user management page.

Within this page, you can invite new users, change roles, and remove members.

Adding a user

To add a new user to our Calendly account, you need to invite them. This is done by navigating to the Admin center's Users page and clicking the Invite users button (around the top-right of the page). Alternatively you can go directly to the Invite user page using this direct link.

On the Invite user page, you will enter the email of the user you wish to add and the group to put them in (if applicable). After doing so, click the Next: Set roles button (at the bottom-right of the page).

You will then select the role for each user you are adding (User or Admin). After doing so, click the Next: Assign event types button (at the bottom-right of the page).

You will then select the default events to have added to the user's profile. Once you have selected the applicable ones, click the Send invitation button (at the bottom-right of the page) to finish the process.

Changing roles

To change the role of a user, locate the user on the Admin center's Users page, click the three vertical dots at the far right of their row, and click the Change Role option.

This will bring show a pop-up form where you can select the new role to use. Once you have done so, click the Apply button.

Removing a user

To remove a user, locate the user on the Admin center's Users page. click the three vertical dots at the far right of their row, and click the Remove option.

This will bring show a pop-up form where you select the deletion level:

- Remove members
- Remove members and delete their accounts

Once you have selected the deletion level, click the Remove button.

Team management

Managing teams in Calendly is done via the team's page in question. To locate this, go to your home page, click the drop-down at the top of the page (it should say My Calendly at the time), and select the team you are wishing to manage (should be under the Teams section).

Adding users to a team

To add a member to a team, click the gear icon to the right of the + New Event Type and then select the Edit Team option.

On this page, click within the Team Members box and start typing out the email of the person you are wishing to add. Click the person in the drop-down that appears from your search.

If the person you are adding should also be able to manage the team, locate them in the user list (below the Team Members box) and click the checkbox next to can manage team.

Once you are done, click the Save button at the bottom of the page.

Removing users from a team

To remove a member from a team, click the gear icon to the right of the + New Event Type and then select the Edit Team option.

On this page, locate the member in question to remove from the list of members (below the Team Members box) and click the Remove link below their name.

Once you are done, click the Save button at the bottom of the page.

Group management

Group management in Calendly is primarily done via the Admin center's Groups page. To access this page after logging in, you would click navigate to the Admin center by clicking the option at the lower-left of the page. After navigating there, click Groups on the left-hand side to get to the group management page.

Creating a group

To create a group, avigate to the Admin center's Groups page and click thhe New Group button (at the top-right of the page).

On the pop-up box that appears, enter the name of your new group and click the Continue button.

Deleting a group

To create a group, avigate to the Admin center's Groups page and locate the group you wish to delete. Click the three vertical dots to the rightht of its row and click the Delete option.

A pop-up box will appear for confirmation. To proceed, type the name of the group in the input box. After doing so, click the Delete group button.

Adding users to a group

To add users to a group, navigate to the Admin center's Groups page, locate the group in question, and click the three vertical dots on the right side of the row. You will then select the Edit option.

From here, click the Add members button (at the top-right of the page), locate the user by typing their name or email in the search box, and click them in the drop-down. Once you have gotten all the users you wish to add selected, click theh Add button.

Removing users to a group

To remove users to a group, navigate to the Admin center's Groups page, locate the group in

question, and click the three vertical dots on the right side of the row. You will then select the Edit option.

From here, locate the user in question from the list of users, click the three vertical dots at the right-side of their row, and click Remove as member. A pop-up box will ask you to confirm the removal. Do so by clicking the Yes, remove button.

Creating an event type

To create a new event type, go to your home page, click the drop-down at the top of the page (it should say My Calendly at the time), and select the person, group, or team you are wishing to create an event type for. You will then click the New Event Type button at the top-left of the page.

On the next page, you will need to determine what kind of event type the meeting is for. The exact options available depend on if you are creating the event type for a person or group or team, but in general the common types are:

- One-on-One: your standard meeting type
- Round Robin: cycles between all available
- Collective: scheduling for when all selected are available (basically group scheduling that takes everyone's availability into effect)
- Group: allows for inviting multiple people to the same meeting

The exact type to use is going to be based on what you are aiming to do.

Once you have decided the event type to use, click the option on the page you are on.

Depending on the type of event, it might ask you to confirm who is hosting the event. Select the person from the drop-down and then click the Next button.

This will then bring up the event creation page, which will have a plethora of options for you to select and use. We highly recommend reading How to set up an event type for more information.

Managing event types

To manage an event type, locate the event type in question and click on the gear icon in the top-right of the box. After doing so, click the Edit option in the sub-menu. Doing so will open the event type editor. From here, there are a lot of options that can be edited.

Webhooks

Creating a webhooks

To create a webhook in Calendly, you have to use the API.

The very first thing you are going to need to do is determine our Organization's information, as it will be required for the webhook's creation.

To do this, you are going to need to use the Get current user API endpoint, like so:

```
bash
curl --request GET \
  --url https://api.calendly.com/users/me \
  --header 'Authorization: Bearer TOKENGOESHERE' \
  --header 'Content-Type: application/json'
```

This will produce output that contains our organization's information (as you are part of our organization). The exact entry you are wanting is the currentorganization value. If using something like jq, you can run the entire command like so:

```
bash
curl -ss --request GET --url https://api.calendly.com/users/me \
  --header 'Authorization: Bearer TOKENGOESHERE' \
  --header 'Content-Type: application/json' \
  | jq '.resource.currentorganization'
```

You will also need your user reference URL, which you can get from the same endpoint. In this case, you'll need uri value from the output. Using something like jq, you can run the entire command like so:

```
bash
curl -ss --request GET --url https://api.calendly.com/users/me \
  --header 'Authorization: Bearer TOKENGOESHERE' \
  --header 'Content-Type: application/json' \
  | jq '.resource.uri'
```

```
{{% alert title="Note" color="primary" %}}
```

If you prefer, you can use the gitlabssupportreadiness gem instead to get all this information:

```
ruby
require 'supportreadiness'
config = Readiness::Calendly::Configuration.new
config.token = 'test123abc'
client = Readiness::Calendly::Client.new(config)
user = Readiness::Calendly::Users.current(client)
puts user.uri
puts user.currentorganization
```

```
{{% /alert %}}
```

With that, you can now create the webhook itself. To do this, you will use the Create Webhook Subscription API endpoint. This is going to require some very specific information to work correct, so let's break it down item by item:

- url
 - This is the URL the payload from Calendly is being sent to.
- events
 - An array of items the webhook will trigger for. The options currently are:
 - invitee.created
 - invitee.canceled
 - inviteenoshow.created
 - routingformsubmission.created
- organization
 - The URL for your organization. See above to get this value.
 - NOTE The full URL should be used.
- user
 - Your user reference URL. See above to get this value
 - NOTE The full URL should be used.
- scope
 - The scope this runs on. It can either be user or organization.

Putting this all together, you will make a POST request, with the data being in JSON format. To accomplish this, we recommend making a JSON file containing the parameters to send, verifying it via jq, and then making your API call. An example of what this might look like is:

```
bash
echo '{' >> temp.json
echo '"url": "https://blah.foo/bar",' >> temp.json
echo '  "events": [' >> temp.json
echo '    "invitee.created",' >> temp.json
echo '    "invitee.canceled",' >> temp.json
echo '    "inviteenoshow.created"' >> temp.json
echo '  ],' >> temp.json
echo '  "organization": "https://api.calendly.com/organizations/AAAAAAAAAAAAAAAAAAAA",' >> temp.json
echo '  "user": "https://api.calendly.com/usersBBBBBBBBBBBBBBBBBBBB",' >> temp.json
echo '  "scope": "user"' >> temp.json
echo '}' >> temp.json
$ cat temp.json | jq
{
  "url": "https://blah.foo/bar",
  "events": [
    "invitee.created",
    "invitee.canceled",
    "inviteenoshow.created"
  ],
  "organization": "https://api.calendly.com/organizations/AAAAAAAAAAAAAAAAAAAA",
  "user": "https://api.calendly.com/usersBBBBBBBBBBBBBBBBBBBB",
  "scope": "user"
}

curl -ss --request POST \
  --url https://api.calendly.com/webhooksubscriptions \
  --header 'Authorization: Bearer TOKENGOESHERE' \
```

```
--header 'Content-Type: application/json' \  
--data @temp.json
```

The response you get back from this need to be verified, but it should mirror a lot of the information you just used in your parameters. Once you have verified it all looks correct, the webhook is live.

Need a list of all Calendly webhooks?

No worries, it happens from time to time. To do this, you will want to use the List Webhook Subscriptions to get that information. This requires you so specify a scope and the organization itself. This might look like (assuming you URL encode the values, which isn't "required" but helpful):

```
bash  
curl -ss --request GET \  
  --url 'https://api.calendly.com/webhooksubscriptions?organization=https%3A%2F%2Fapi.calendly.com%2Forganizations%2FAAAAAAAAAAAAAAAAAAAAA&scope=organization' \  
  --header 'Authorization: Bearer TOKENGOESHERE' \  
  --header 'Content-Type: application/json'
```

Calendly Event to Google Calendar Event

Both our Global and US Government support teams have a desire for events booked via their Calendly links to populate specific team-wide calendars instead of just their own. To accomplish this, we have setup a combination of Calendly webhooks and our own GitLab CI/CD.

This is centered around the Calendly Events to gCal Events project.

How it works

A payload is sent from Calendly via a gitlab.com webhook to the project. The script then processes this payload to determine the parameters of the Google calendar event to create (if one is to be created at all).

```
---  
## SECTION: security/customer-support-operations/docs/gitlab/_index.md
```

```
---  
## SECTION: security/customer-support-operations/docs/gitlab/cmpps.md
```

^{Introduced via support-team-meta4531}

```
{{% pageinfo color="warning" %}}
```


This is an information page for contact management projects.

If you are looking for information about managing contact management projects, please see workflows.

{{% /pageinfo %}}

What is a contact management project?

Contact management projects are projects on GitLab.com that allows a customer to manage their support contacts on Zendesk Global.

How does it work

When commits are made on the project in question, a webhook triggers with specific variables to trigger a pipeline on the contacts project.

The code itself parses the YAML file for the project that triggered the pipeline to determine what organization contact changes are required on Zendesk Global.

It will then perform a sync with the needed changes.

Depending on the results, a badge will be added to the project. This is to help the project developers know the current status of the sync.

SECTION: security/customer-support-operations/docs/gitlab/gl-support-bot-admin.md

What is gl-support-bot used for?

For many of our scripts and automations that require admin level permissions, we use tokens owned by the gl-support-bot-admin user. This is to ensure our automations are not tied to a specific human user and run consistently.

List of usage

For scalability reasons, we cannot list all tokens available. Please log in as the user and review them via the Personal Access Tokens page.

Requesting a token

{{% alert title="Note" color="danger" %}}

This bot account has access to a lot of data. Unless being used in the standard methods we utilize, you should consider using an alternative source (such as service accounts).

{{% /alert %}}

To request the token, file an access request.

For the issue, the Service Name is gl-support-bot-admin

Please assign the issue to Jason Colyer and Lyle Kozloff at this time.

SECTION: security/customer-support-operations/docs/gitlab/gl-support-bot.md

What is gl-support-bot used for?

For many of our scripts and automations, we use tokens owned by the gl-support-bot user. This is to ensure our automations are not tied to a specific human user and run consistently.

List of usage

For scalability reasons, we cannot list all tokens available. Please log in as the user and review them via the Personal Access Tokens page.

Requesting a token

{{% alert title="Note" color="danger" %}}

This bot account has access to a lot of data. Unless being used in the standard methods we utilize, you should consider using an alternative source (such as service accounts).

{{% /alert %}}

To request the token, file an access request.

For the issue, the Service Name is gl-support-bot

Please assign the issue to Jason Colyer and Lyle Kozloff at this time.

SECTION: security/customer-support-operations/docs/gitlab/investigations.md

Introduction

As we look at different ways of doing things, our operating question should be "How?". An investigation issue is an opportunity to come up with creative solutions to solving the challenging problems our business faces.

The outcome of an investigation issue is an artifact that will help a DRI make an informed decision on a topic where there may be tradeoffs or nuance involved.

Performing an investigation

There's no set way to go about this, but rather a loose set of guidelines.

1. Right size your effort: the idea isn't to solve the problem but build personal expertise and a portfolio of possible solutions.
2. Consider commercial solutions: development and maintenance costs are not always worth it! If there's an off-the-shelf solution, it should be considered.
3. Consider Dogfooding: if there's a feature of our product that can be used it should be seriously considered.
4. Document critical blockers early: If you discover a fundamental issue that would prevent a particular approach from working, document it immediately rather than continuing down that path.
5. Consider sustainability: Evaluate how solutions will scale with increased load and what maintenance burden they'll create over time. Factor in dependencies on third-party systems and their stability.
6. Consult stakeholders: Talk to the people who will be using or affected by the solution. Their insights into daily workflows and pain points are invaluable.
7. Consider boring solutions: Favor proven, simple approaches over novel or complex ones. The most reliable solutions are often those with established track records, even if they seem less exciting. Balance implementation timeline, security requirements, and resource constraints accordingly.

Deliverables

A good engineering investigation should produce:

1. Executive Summary: A concise overview highlighting key findings and recommendations.
2. Approach Analysis: Evaluation of each potential solution including pros, cons, and resource requirements.
3. Recommendation: A clear suggestion for the path forward with supporting rationale.
4. Implementation Considerations: High-level notes on how the recommended approach would be executed.
5. Open Questions: Any unresolved issues that require further investigation.

Remember, the goal is to provide decision-makers with the information they need to choose the best path forward, not to make the final decision yourself at the point of the investigation.

Example Template

The following is just an example template to help you get started. Please modify it as is appropriate to the situation.

markdown

Engineering Investigation / Research Spike - Template: Modify as needed

Executive Summary

[1-2 paragraphs summarizing the investigation, key findings, and critical concerns. Highlight any significant risks or "dealbreakers" that would make certain approaches unviable.]

Critical Issues Alert: [Highlight any showstopper issues in a distinct section]

Problem Statement

[Clearly define the problem being solved and why it matters]

- Current Situation: [Describe the current state]
- Goals: [What we're trying to accomplish]
- Success Metrics: [How we'll measure success]
- Stakeholders: [Who is affected by this decision]

Commercial Solutions

[Evaluate existing products or services that could address the problem]

Solution 1: [Product/Vendor Name]

Description: [Brief explanation of the product and how it addresses our needs]

Features:

- [Key feature 1]
- [Key feature 2]
- [Key feature 3]

Pricing Model:

- [License costs, subscription fees, or pricing tiers]
- [Implementation or professional services costs]
- [Ongoing maintenance or support costs]

Pros:

- [Benefit 1]
- [Benefit 2]

Cons:

- [Drawback 1]
- [Drawback 2]

Vendor Assessment:

- Company Stability: [Assessment of vendor's market position and longevity]
- Support Quality: [Information about support options and responsiveness]
- Integration Capabilities: [How well it integrates with existing systems]
- Customer References: [Feedback from other customers if available]

Solution 2: [Product/Vendor Name]

[Same structure as Solution 1]

Custom Approach Options

Option 1: [Name of Approach]

Description: [Brief explanation of what this approach entails]

Implementation Details:

- [Key technical components required]
- [Integration points]
- [Architecture considerations]

Pros:

- [Benefit 1]
- [Benefit 2]
- [Benefit 3]

Cons:

- [Drawback 1]
- [Drawback 2]
- [Drawback 3]

Resource Requirements:

- Estimated Development Effort: [Low/Medium/High or story points/time]
- Maintenance Overhead: [Low/Medium/High]
- Dependencies: [Other systems, teams, or external factors]

Risks:

- [Risk 1: Impact and mitigation strategy]
- [Risk 2: Impact and mitigation strategy]

Option 2: [Name of Approach]

[Same structure as Option 1]

Option 3: [Name of Approach]

[Same structure as Option 1]

Comparative Analysis

Criteria	Commercial Solution 1	Commercial Solution 2	Custom Option 1	Custom Option 2	Custom Option 3
Development Time	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
Maintenance Complexity	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
Scalability	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
Performance	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
User Experience	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
Security	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]
Cost	[Rating]	[Rating]	[Rating]	[Rating]	[Rating]

Recommendation

[Clear statement of recommended approach with justification]

Implementation Roadmap

[If applicable, high-level steps for implementing the recommended approach]

1. [Step 1]
2. [Step 2]
3. [Step 3]

Open Questions

[List any unresolved questions that need further investigation]

Appendix

[Any supporting information, diagrams, code examples, or references]

SECTION: security/customer-support-operations/docs/gitlab/issue-boards.md

We use issues boards to get a quick overview of our workload. Here we detail the primary ones we use.

Stages

Located here

This shows all non-closed issues in groupings of the stage they are in.

Customer

Located here

This shows all non-closed issues groupings of the customer team that requested them.

Request Types

Located here

This shows all non-closed issues groupings of the type of request.

SECTION: security/customer-support-operations/docs/gitlab/labels.md

Stage labels

These are scoped labels, meaning only one of them can ever be present on an issue at a time. This is because an issue cannot be in two stages at once.

Label	What it means
-----	-----
Stage::Triage	This is where i